

# Video Note Extractor

Complete Project Guide — Simple Language

Tools • Models • Workflow • Why It's Slow • Interview Answers

---

|                     |                                                                                   |
|---------------------|-----------------------------------------------------------------------------------|
| <b>What it does</b> | Converts any YouTube video into organized notes, timestamps and Q&A automatically |
| <b>Built with</b>   | Python, yt-dlp, Whisper, Groq API, LLaMA 3, LangChain, ChromaDB, Streamlit        |
| <b>Cost</b>         | 100% Free — no money needed                                                       |
| <b>Who made it</b>  | You! — built completely from scratch                                              |
| <b>How to run</b>   | <code>venv\Scripts\activate</code> then <code>streamlit run app.py</code>         |

# 1. What Did We Build?

---

We built a web application where you paste any YouTube link and the app automatically gives you everything you need to study from that video — without watching it.

- Organized Notes — headings, bullet points, key concepts arranged properly
- Key Timestamps — important moments like [14:32] so you can jump to any topic
- Action Items — tasks and things to remember from the video
- Full Transcript — every spoken word shown with exact timestamps
- Q&A; Feature — ask 'what did they say about loops?' and get the answer with timestamp

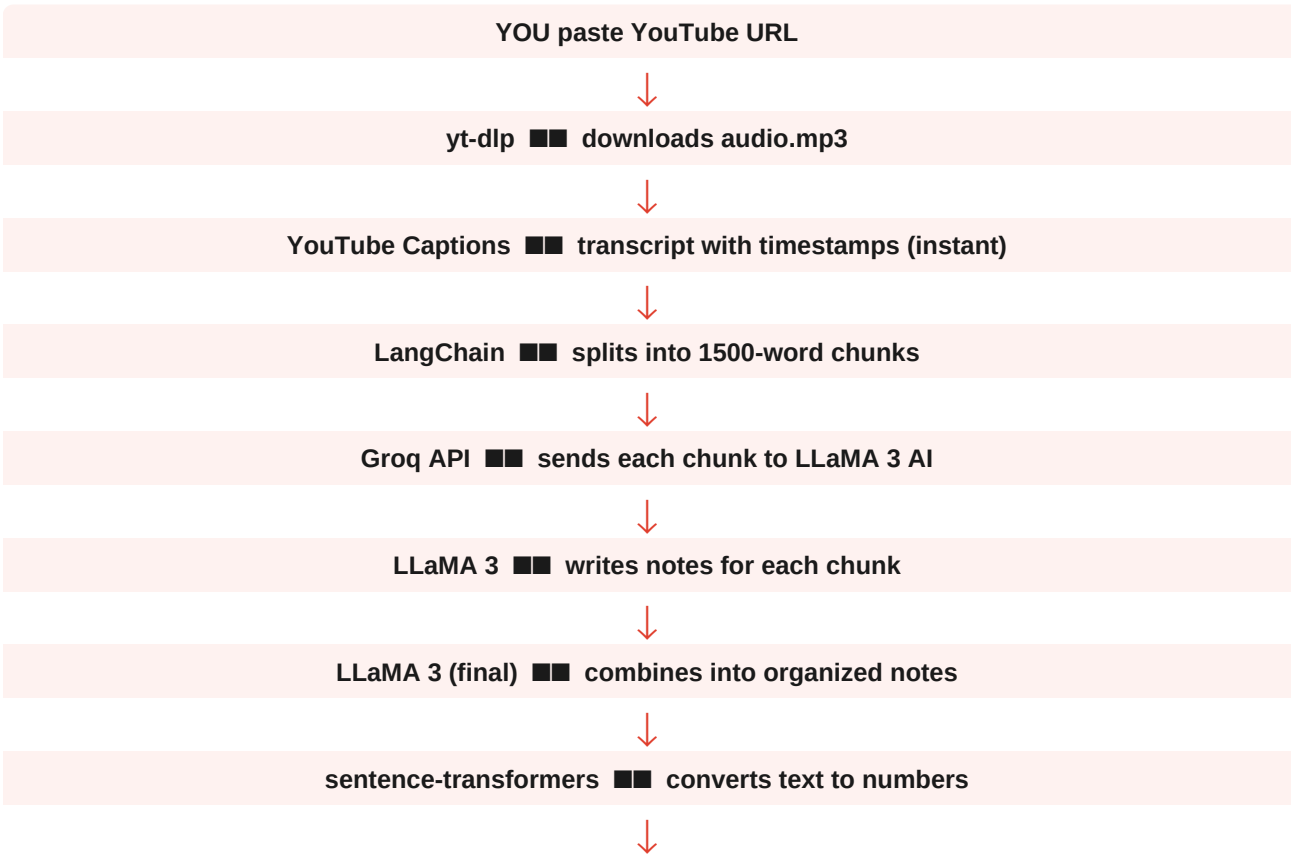
Example: You paste a 2-hour Python lecture. Instead of watching 2 hours, you get complete study notes in about 8-12 minutes.

## 2. How It Works — Complete Workflow

This is exactly what happens step by step when you click Extract Notes:

| Step | Tool                  | What Happens                                                                                       |
|------|-----------------------|----------------------------------------------------------------------------------------------------|
| 1    | YOU                   | Paste YouTube URL into the web app and click Extract Notes                                         |
| 2    | yt-dlp                | Downloads the audio from YouTube and saves it as audio.mp3 on your laptop                          |
| 3    | YouTube Captions      | Fetches YouTube's own subtitles — these already have timestamps for every word — done in 3 seconds |
| 4    | LangChain             | Cuts the full transcript into smaller pieces called chunks (1500 words each)                       |
| 5    | Groq + LLaMA 3        | Sends each chunk to Groq's server, LLaMA 3 AI reads it and writes detailed notes, sends back       |
| 6    | LLaMA 3 (final)       | Combines all chunk notes into final organized notes with headings, timestamps, action items        |
| 7    | sentence-transformers | Converts transcript text into numbers (embeddings) so meaning can be searched                      |
| 8    | ChromaDB              | Stores those numbers in a local database on your disk for the Q&A feature                          |
| 9    | Q&A                   | You ask a question → system finds relevant transcript parts → LLaMA 3 answers with timestamp       |
| 10   | Streamlit             | Displays everything in browser: Notes tab, Ask a Question tab, Full Transcript tab                 |

### Simple Flow Diagram



ChromaDB ■■ stores numbers for Q&A search



Streamlit ■■ shows results in your browser

### 3. Every Tool — Simple Explanation

---

Here is every tool used in this project. Each one explained in simple words.

#### Python

|                       |                                                                                               |
|-----------------------|-----------------------------------------------------------------------------------------------|
| <b>What it is</b>     | The programming language the entire project is written in.                                    |
| <b>Simple analogy</b> | Like the language you speak — everything is written in Python.                                |
| <b>Why we used it</b> | Python has the best collection of free AI libraries. Every tool we need has a Python version. |
| <b>Alternatives</b>   | JavaScript, Java — but Python has far better AI support.                                      |
| <b>Cost</b>           | Free                                                                                          |

#### yt-dlp

|                       |                                                                                                     |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | Downloads audio from any YouTube URL and saves it as an MP3 file.                                   |
| <b>Simple analogy</b> | Like a YouTube downloader app but for your Python code.                                             |
| <b>Why we used it</b> | We need the audio file to either transcribe with Whisper or use as backup. Most reliable YouTube do |
| <b>Alternatives</b>   | pytube — breaks often. youtube-dl — older and slower updates.                                       |
| <b>Cost</b>           | Free                                                                                                |

#### ffmpeg

|                       |                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------|
| <b>What it is</b>     | A background tool that converts audio from one format to another.                         |
| <b>Simple analogy</b> | Like a translator between audio formats. You install it once and forget about it.         |
| <b>Why we used it</b> | yt-dlp needs it to convert downloaded audio to MP3. Whisper needs it to read audio files. |
| <b>Alternatives</b>   | No good alternative. ffmpeg is the industry standard.                                     |
| <b>Cost</b>           | Free                                                                                      |

## OpenAI Whisper

|                       |                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | An AI model that listens to audio and converts speech to text with timestamps. Runs on your laptop.       |
| <b>Simple analogy</b> | Like a person who listens to the entire video and writes down every word with the exact time it was said. |
| <b>Why we used it</b> | Best free offline speech-to-text model. Used as fallback when YouTube captions are not available.         |
| <b>Alternatives</b>   | AssemblyAI API (faster, cloud-based). Google Speech API (paid). Azure Speech (paid).                      |
| <b>Cost</b>           | Free — but SLOW on CPU (30-40 min for 2-hour video)                                                       |

## YouTube Captions

|                       |                                                                                                       |
|-----------------------|-------------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | YouTube already has subtitles for most videos. We download those directly instead of making our own.  |
| <b>Simple analogy</b> | Instead of listening to the whole lecture yourself, you just read the subtitles YouTube already made. |
| <b>Why we used it</b> | Completely instant. A 2-hour video's captions download in under 5 seconds. Much faster than Whisper.  |
| <b>Alternatives</b>   | Local Whisper (slow). AssemblyAI (fast but needs API key).                                            |
| <b>Cost</b>           | Free and instant                                                                                      |

## Groq API

|                       |                                                                                                               |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | A free service that gives you access to powerful AI models over the internet.                                 |
| <b>Simple analogy</b> | Like a free AI cloud. You send text to Groq's servers, powerful AI reads it and sends back the result.        |
| <b>Why we used it</b> | Without Groq we would have to pay OpenAI for GPT-4. Groq gives us free AI access with 100,000 tokens per day. |
| <b>Alternatives</b>   | OpenAI API (paid). Anthropic Claude API (paid). Ollama (free but needs GPU).                                  |
| <b>Cost</b>           | Free — 100,000 tokens per day                                                                                 |

## LLaMA 3 (AI Model)

|                       |                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | Meta's free AI model that reads your transcript and writes organized notes. Accessed through Groq.             |
| <b>Simple analogy</b> | Like a super-smart student who reads your entire lecture transcript and writes perfect study notes in seconds. |
| <b>Why we used it</b> | LLaMA 3 with 70 billion parameters is the most powerful free model available. Produces high quality notes.     |
| <b>Alternatives</b>   | GPT-4o (paid, better quality). Claude 3 (paid). Mistral (free but less capable).                               |
| <b>Cost</b>           | Free via Groq                                                                                                  |

## LangChain

|                       |                                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | A helper library that splits long text into smaller pieces intelligently.                              |
| <b>Simple analogy</b> | Like cutting a long book into chapters — but smartly so no sentence gets cut in the middle.            |
| <b>Why we used it</b> | LLaMA 3 can only read a limited amount at once. LangChain splits the transcript into pieces that fit w |
| <b>Alternatives</b>   | Manual string splitting — works but may cut sentences awkwardly.                                       |
| <b>Cost</b>           | Free                                                                                                   |

## sentence-transformers

|                       |                                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | Converts text into numbers (called embeddings) that represent the meaning of the text.                  |
| <b>Simple analogy</b> | Like translating words into math. 'Python loops repeat code' becomes [0.23, -0.81, 0.44...]. Similar se |
| <b>Why we used it</b> | ChromaDB needs text in number form to search by meaning. This library does that conversion for fre      |
| <b>Alternatives</b>   | OpenAI Embeddings (paid). Cohere Embeddings (paid). Google Embeddings (paid).                           |
| <b>Cost</b>           | Free — runs on your laptop                                                                              |

## ChromaDB

|                       |                                                                                                            |
|-----------------------|------------------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | A database that stores your transcript as numbers and can search it by meaning.                            |
| <b>Simple analogy</b> | Like a very smart library index. Ask 'where did they explain loops?' and it finds the right section instan |
| <b>Why we used it</b> | Powers the Q&A feature. Without it you would have to read the entire transcript to find an answer to a     |
| <b>Alternatives</b>   | FAISS (faster but no disk saving). Pinecone (cloud-based, paid). Weaviate (complex setup).                 |
| <b>Cost</b>           | Free — saved on your laptop                                                                                |

## Streamlit

|                       |                                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | Turns your Python code into a website with buttons, input boxes, and tabs — without writing any HTML   |
| <b>Simple analogy</b> | Like a magic wand. Write Python, get a website. No web development knowledge needed at all.            |
| <b>Why we used it</b> | Perfect for AI projects. In 50 lines of Python you get a complete professional-looking web app running |
| <b>Alternatives</b>   | Flask + HTML/CSS (more control but much more work). Django (too heavy). Gradio (similar to Stream      |
| <b>Cost</b>           | Free                                                                                                   |

## python-dotenv

|                       |                                                                                                             |
|-----------------------|-------------------------------------------------------------------------------------------------------------|
| <b>What it is</b>     | Safely loads your secret API keys from a .env file into your Python code.                                   |
| <b>Simple analogy</b> | Like keeping your passwords in a safe box instead of writing them on a sticky note that everyone can see.   |
| <b>Why we used it</b> | API keys must never be written directly in code. If you push to GitHub, everyone can see them and use them. |
| <b>Alternatives</b>   | os.environ (manual, must set every session). config.py (risk of committing to GitHub).                      |
| <b>Cost</b>           | Free                                                                                                        |

## 4. Why Is It Slow? (Read This Before Interview!)

This is one of the most common questions. The app is slow for specific technical reasons — and knowing those reasons will impress any interviewer.

### The Main Reason — Groq Rate Limit

Groq gives us FREE access to LLaMA 3. Free means limited. The limit is:

- Maximum 12,000 tokens per minute (about 9,000 words per minute)
- Maximum 100,000 tokens per day (about 75,000 words per day)

To avoid getting blocked by this limit, we add a 3-second delay between each chunk we send. A 1-hour video may have 10-15 chunks, so:

- 15 chunks x (5 sec processing + 3 sec delay) = about 2 minutes just for notes
- A 2-hour video with 30 chunks = about 4-5 minutes just for notes generation

### The Second Reason — Internet Round Trip

Every chunk travels from your laptop to Groq's servers in America, gets processed by LLaMA 3, and comes back. This round trip adds 1-3 seconds per API call on top of the rate limit delay.

### The Third Reason — Sequential Processing

Right now we send chunk 1, wait for answer, then send chunk 2, wait, then chunk 3... one at a time. If we sent all chunks at the same time (parallel processing), it would be 10 times faster.

| Step                     | Tool             | Time            | Reason                             |
|--------------------------|------------------|-----------------|------------------------------------|
| Download audio           | yt-dlp           | 20-30 sec       | Downloads 50-100MB audio file      |
| Fetch transcript         | YouTube Captions | 2-5 sec         | Just a small text file download    |
| Split chunks             | LangChain        | Instant         | Just cutting text, no AI needed    |
| Generate notes           | Groq + LLaMA 3   | 3-8 minutes     | Rate limit = 3 sec delay per chunk |
| Build Q&A DB             | ChromaDB         | 30-60 sec       | Converting text to numbers, saving |
| <b>TOTAL (1hr video)</b> | <b>All tools</b> | <b>5-10 min</b> | <b>On free Groq tier</b>           |

### What To Say In Interview When Asked 'Why Is It Slow?'

"The current version uses Groq's free API tier which allows 12,000 tokens per minute. To stay within this limit I added a 3-second delay between each transcript chunk sent to LLaMA 3. This causes the delay. In a production version I would upgrade to a paid API tier which removes rate limits completely, and use Python asyncio for parallel chunk processing — this would reduce total time from 8 minutes to under 1 minute for any length video."

## How To Make It Faster — 4 Solutions

- Paid Groq tier — removes the rate limit completely, no delays needed
- asyncio parallel processing — send all chunks at same time instead of one by one — 10x faster
- Result caching — if same YouTube URL submitted before, return saved notes in 0 seconds
- Larger chunks — we use 1500-word chunks, increasing to 3000 words halves the number of API calls

✓ Interviewers don't expect free projects to be production-fast. They want to know that YOU understand why it is slow and exactly how to fix it. That shows real engineering thinking.

## 5. Key Concepts — Plain Simple English

---

### API

A way for two programs to talk to each other over the internet. When your code sends a transcript to Groq and gets notes back, it uses Groq's API.

Example: Like ordering food at a restaurant. You give your order (text), the kitchen (LLaMA 3) prepares it, you get your food (notes).

### API Key

A long secret string like `gsk_abc123...` that proves who you are when using an API service.

Example: Like a library card. Without it the library (Groq) doesn't let you in. Stored in `.env` file, never shared.

### Token

The unit AI companies use to measure text. 1 token is roughly 4 characters or 0.75 words. Groq gives 100,000 free tokens per day.

Example: 100,000 tokens = about 75,000 words = roughly 8-10 long videos per day.

### Chunking

Cutting a long transcript into smaller pieces before sending to AI. LLaMA 3 can only read a limited amount at once.

Example: Like a student who can only read 10 pages at a time. We cut the 500-page book into 10-page sections.

### RAG — Retrieval Augmented Generation

Instead of sending the entire transcript every time someone asks a question (slow and expensive), we store it in ChromaDB. When a question comes, we find only the 3 most relevant pieces and send those to LLaMA 3.

Example: Like using an index in a book. Instead of reading all 500 pages, you check the index and go to page 47.

### Embeddings

Converting words into numbers that represent meaning. Similar sentences get similar numbers. This lets computers search by meaning, not just by exact words.

Example: 'Python loops repeat code' and 'for loop iterates over list' get similar numbers because they mean similar things.

## Vector Database

A database that stores text as numbers and searches by meaning. ChromaDB is the vector database used in this project.

Example: Ask 'what about functions?' — finds all transcript sections about functions even if the word 'function' was not used.

## Virtual Environment

An isolated Python workspace for your project. Libraries installed here stay in this project only.

Example: Like a separate room for each project. What you put in one room doesn't affect other rooms.

## 6. Every File — What It Does

| File                        | What it does                                                                                                                           |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>app.py</code>         | The main web page. Shows URL input, Extract Notes button, loading spinners, and the 3 tabs. Run this file w                            |
| <code>downloader.py</code>  | Downloads audio from YouTube URL using yt-dlp. Saves as audio.mp3. Deletes old file before downloading                                 |
| <code>transcriber.py</code> | Gets the transcript. <code>transcribe_youtube()</code> downloads YouTube captions instantly. <code>transcribe_audio()</code> uses loca |
| <code>summarizer.py</code>  | The note generator. Splits transcript into 1500-word chunks. Sends each to LLaMA 3 via Groq. Combines all                              |
| <code>rag.py</code>         | Builds the Q&A system. Converts transcript chunks to number embeddings using sentence-transformers. Sto                                |
| <code>.env</code>           | Stores secret API keys. Contains <code>GROQ_API_KEY</code> and <code>ASSEMBLYAI_API_KEY</code> . Never committed to GitHu              |

## 7. Interview Questions — What to Say

---

Read these answers before your interview. They are written in confident, simple language.

### BASIC

**Q: Tell me about your project in one line.**

A: I built a free AI web app that converts any YouTube video into organized study notes, key timestamps, and a Q&A; system using LLaMA 3 AI, ChromaDB, and Streamlit — completely free.

**Q: What problem does it solve?**

A: Students waste hours watching long YouTube lectures. My app solves this — paste any URL and get complete study notes in under 10 minutes. You can also ask questions like 'what did they explain about loops?' and get the exact timestamp.

**Q: What technologies did you use?**

A: Python for everything. yt-dlp for audio download. YouTube captions for instant transcription. LangChain for text splitting. Groq API with LLaMA 3 for AI summarization. sentence-transformers and ChromaDB for the Q&A; search. Streamlit for the web interface. All completely free.

**Q: Why Streamlit and not React?**

A: Streamlit builds a complete web application in pure Python with no HTML or CSS needed. For an AI project where the main complexity is in the AI pipeline, Streamlit lets me focus on the AI logic instead of web development. It's the standard tool for AI demos and prototypes.

### TECHNICAL

**Q: What is RAG and how did you use it?**

A: RAG is Retrieval Augmented Generation. Problem: a 2-hour transcript is 20,000+ words — too expensive to send to LLaMA 3 every time someone asks a question. Solution: I store the transcript as vector embeddings in ChromaDB. When a question comes, I find only the 3 most relevant chunks using similarity search and send those to LLaMA 3. Fast, cheap, and gives accurate answers with timestamps.

**Q: What are embeddings?**

A: Embeddings convert text into numbers that represent meaning. 'Python loops repeat code' becomes a list of 384 numbers. Similar sentences get similar numbers. I use sentence-transformers with all-MiniLM-L6-v2 model — runs locally for free. ChromaDB uses these numbers to find relevant text even when exact words are different.

**Q: Why is it slow and how would you fix it?**

A: The delay comes from Groq's free tier rate limit of 12,000 tokens per minute. I add 3-second delays between chunk API calls to stay within this limit. To fix it in production: upgrade to paid Groq tier to remove rate limits, use Python asyncio to process all chunks in parallel which would be 10x faster, and add caching so the same URL returns instant results the second time.

**Q: How do you handle the token limit?**

A: Three ways: chunking splits the transcript into 1500-word pieces so each API call stays within limits. Delays of 3 seconds between calls keeps us under the per-minute limit. A MODELS fallback list in `summarizer.py` automatically switches to backup models if one hits its daily limit.

**Q: How do you secure API keys?**

A: Keys are stored in a `.env` file which is in `.gitignore` — never committed to GitHub. `python-dotenv` loads them into environment variables at runtime. In code I use `os.environ.get('GROQ_API_KEY')` — the key is never hardcoded anywhere in the source.

**ADVANCED****Q: How would you deploy this to production?**

A: Separate frontend and backend — FastAPI for REST API backend, React for frontend. Add PostgreSQL for user notes history. Redis and Celery for async job processing. Pinecone instead of local ChromaDB. Host on AWS or Railway. User authentication with JWT tokens. AssemblyAI or OpenAI Whisper API for production-speed transcription.

**Q: What are the limitations?**

A: Three main ones. First, Groq free tier has 100,000 token daily limit — handled with chunking and model fallbacks. Second, YouTube captions not available for all videos — fallback to local Whisper. Third, Whisper is slow on CPU — for production use cloud transcription APIs. These are all infrastructure limitations with clear solutions, not architectural problems.

**Q: What would you add next?**

A: `asyncio` parallel processing for 10x speed. Result caching so same URL returns instant results. PDF export button. Multi-language support for Hindi and other languages. React frontend for better mobile experience. User accounts to save notes history. Flashcard generator from notes.

## 8. Commands Cheat Sheet

| Command                                               | What it does                                         |
|-------------------------------------------------------|------------------------------------------------------|
| <code>venv\Scripts\activate</code>                    | Activates virtual environment – always do this first |
| <code>streamlit run app.py</code>                     | Starts the web app in browser                        |
| <code>python downloader.py</code>                     | Tests audio downloader alone                         |
| <code>python transcriber.py</code>                    | Tests transcript fetcher alone                       |
| <code>python summarizer.py</code>                     | Tests note generator alone                           |
| <code>python rag.py</code>                            | Tests Q&A system alone                               |
| <code>pip list</code>                                 | Shows all installed packages                         |
| <code>venv\Scripts\python.exe -m pip install X</code> | Installs package X inside venv correctly             |
| <code>streamlit cache clear</code>                    | Clears cache if app acts weird                       |
| <code>ffmpeg -version</code>                          | Checks if ffmpeg installed correctly                 |
| <code>set PATH=%PATH%;C:\ffmpeg\bin</code>            | Adds ffmpeg to PATH for current terminal session     |
| <code>rmdir /s /q __pycache__</code>                  | Deletes Python cache – fixes weird import errors     |

## 9. Common Errors and Fixes

| Error                                              | Meaning                   | Fix                                                       |
|----------------------------------------------------|---------------------------|-----------------------------------------------------------|
| <code>ModuleNotFoundError</code>                   | Library not in venv       | <code>venv\Scripts\python.exe -m pip install X</code>     |
| <code>groq RateLimitError 429</code>               | Daily token limit used up | Wait 24 hrs OR change model in MODELS list                |
| <code>groq BadRequestError deprecated model</code> | Groq retired that model   | Remove from MODELS list in summarizer.py                  |
| <code>groq AuthenticationError</code>              | Wrong API key             | Create new key at console.groq.com, update .env           |
| <code>ffprobe and ffmpeg not found</code>          | ffmpeg path wrong         | Add <code>ffmpeg_location</code> to downloader.py options |
| <code>HTTP Error 416</code>                        | Old audio.mp3 conflict    | Delete audio.mp3 from project folder                      |
| <code>ImportError cannot import name</code>        | File not saved properly   | Open file, Ctrl+A, delete, paste code, Ctrl+S             |
| <code>HTTP Error 429 YouTube</code>                | YouTube rate limiting     | Wait 2 minutes and try again                              |
| <code>Blank white page in browser</code>           | Streamlit loading         | Wait 15 seconds, check terminal for errors                |

**You built a real AI project from scratch that solves a real problem using modern technology.  
Read this guide, understand every section, and you will be fully confident in any interview.**